

Programmation JAVA : 3 – L2 UPEC

TP 1 : Représentation et affichage de formes géométriques

L'objectif de ce TP est de partir d'un code fonctionnel mais peu extensible, puis de le faire évoluer progressivement vers une conception objet plus propre. On travaillera en particulier les **objets et références**, l'**encapsulation**, les **interfaces**, le **polymorphisme** et la séparation entre **métier** et **affichage**.

Ce premier TP sert également à prendre en main les outils : organisation des fichiers, ouverture d'un projet dans VS Code, compilation, exécution et lecture des messages d'erreur.

Organisation du projet dans VS Code

Le projet est organisé en trois répertoires :

```
code/  
|-- etape1_simple/  
|-- etape2_formes/  
`-- etape3_affichage/
```

Important : Lorsque vous changez d'étape, vous recopiez vous-mêmes les fichiers que vous venez de terminer puis vous les faites évoluer.

Dans VS Code.

- Ouvrez le dossier racine du TP avec **File > Open Folder...**
- N'ouvrez pas simplement un fichier `.java` isolé.
- Travaillez dans le répertoire correspondant à l'étape en cours.
- Utilisez le bouton **Run** lorsqu'il est disponible, ou le terminal.
- En cas d'erreur, commencez par lire le **premier** message d'erreur.

Depuis le terminal. Placez-vous dans le répertoire de l'étape puis utilisez :

```
javac *.java  
java Main
```

À l'étape 3, le programme principal s'appelle `MainSwing` :

```
javac *.java  
java MainSwing
```

Étape 1 – Classes simples et version naïve

Dans cette première étape, vous allez manipuler deux formes géométriques simples : un rectangle et un cercle. L'objectif est de compléter des classes qui représentent ces objets puis de calculer et d'afficher leurs caractéristiques.

Formules utiles.

- Surface d'un rectangle : $S = \text{largeur} \times \text{hauteur}$.
- Périmètre d'un rectangle : $P = 2(\text{largeur} + \text{hauteur})$.
- Surface d'un cercle : $S = \pi r^2$.
- Périmètre d'un cercle : $P = 2\pi r$.

En Java, la constante π est disponible avec `Math.PI`.

1. Premier lancement. Ouvrez le répertoire `etape1_simple`. Les fichiers contiennent des `// TODO`. Commencez par les repérer avant de modifier le code.

Le programme principal contient volontairement une collection de type `Object[]` et utilise `instanceof` pour distinguer les formes :

```

1 for (Object forme : formes) {
2     if (forme instanceof Rectangle r) {
3         // traitement du rectangle
4     } else if (forme instanceof Cercle c) {
5         // traitement du cercle
6     }
7 }
```

Cette version est volontairement naïve. Nous la ferons évoluer à l'étape 2.

2. Classe Point. Complétez `Point.java` :

1. le constructeur ;
2. les getters `getX()` et `getY()` ;
3. la méthode `toString()` afin d'obtenir une représentation de la forme `(x,y)`.

3. Classes Rectangle et Cercle. Pour chacune des deux classes :

1. complétez le constructeur ;
2. complétez les getters ;
3. implémentez `getSurface()` ;
4. implémentez `getPerimetre()` ;
5. redéfinissez `toString()` pour obtenir une représentation lisible.

Compilez régulièrement, plutôt que d'attendre d'avoir rempli tous les `TODO`.

4. Garantir des objets valides dès la construction. Un constructeur ne sert pas uniquement à affecter les attributs : il doit aussi empêcher la création d'un objet incohé-

rent.

Dans ce TP, on adopte les règles suivantes :

- la largeur et la hauteur d'un rectangle doivent être **strictement positives** ;
- le rayon d'un cercle doit être **strictement positif** ;
- les coordonnées d'un point peuvent être négatives : aucune vérification n'est demandée dans `Point`.

Modifiez les constructeurs de `Rectangle` et `Cercle` afin de lever une `IllegalArgumentException` accompagnée d'un message explicite lorsqu'un paramètre ne respecte pas ces règles.

Testez ensuite vos vérifications. Ajoutez **temporairement** dans `Main.java` une construction invalide, par exemple :

```
1 new Cercle(new Point(0, 0), -5);
```

Compilez puis exécutez le programme. Observez le message d'erreur et vérifiez que l'exécution s'interrompt au moment de la construction du cercle.

Faites de même, **un cas à la fois**, avec quelques valeurs invalides :

```
1 new Rectangle(new Point(0, 0), -10, 20);
2 new Rectangle(new Point(0, 0), 10, 0);
3 new Cercle(new Point(0, 0), 0);
```

Comme `IllegalArgumentException` est une exception non vérifiée (une `RuntimeException`), vous n'avez pas besoin d'ajouter de `throws` à la signature du constructeur. Dans ce TP, on laisse volontairement l'exception remonter jusqu'au programme principal : aucun traitement de l'exception n'est demandé.

Une fois les tests terminés, retirez les constructions invalides afin de pouvoir poursuivre le TP normalement.

À retenir : après le retour normal d'un constructeur, l'objet doit être dans un état valide. Si les arguments sont invalides, le constructeur interrompt la construction en levant une `IllegalArgumentException`. Cette garantie devra être conservée lorsque le code sera refactorisé dans les étapes suivantes.

Étape 2 – Introduire un type commun : `Forme`

Nous voulons maintenant supprimer la logique qui consiste à manipuler des `Object` puis à tester successivement leurs types.

1. Préparer l'étape 2. Dans VS Code, copiez depuis `etape1_simple` vers `etape2_formes` les trois fichiers que vous venez de terminer :

`Point.java`
`Rectangle.java`
`Cercle.java`

Ne copiez pas Main.java. Le répertoire `etape2_formes` contient déjà un nouveau programme principal adapté à cette étape.

Vous devez maintenant avoir :

```

etape2_formes/
|-- Point.java          <- votre fichier de l'étape 1
|-- Rectangle.java      <- votre fichier de l'étape 1
|-- Cercle.java         <- votre fichier de l'étape 1
|-- Forme.java          <- fourni
`-- Main.java           <- fourni

```

2. Définir l'interface Forme. Complétez `Forme.java`. Une forme doit fournir :

```

1 double getSurface();
2 double getPerimetre();

```

3. Faire évoluer les classes existantes. Modifiez ensuite `Rectangle` et `Cercle` pour qu'ils implémentent l'interface `Forme`.

Ajoutez également `@Override` devant les méthodes qui implémentent les opérations déclarées dans l'interface.

4. Utiliser le polymorphisme. Complétez le nouveau `Main.java`. Cette fois, la collection doit contenir des `Forme` et non des `Object`.

Le programme principal doit :

1. créer un rectangle et un cercle ;
2. les placer dans une `List<Forme>` ;
3. parcourir la liste avec une boucle `for` ;
4. afficher chaque forme, sa surface et son périmètre ;
5. ne contenir **aucun** `instanceof`.

À observer. Dans la boucle, la variable peut être de type `Forme`. Pourtant, lorsque vous appelez `getSurface()` ou `getPerimetre()`, la méthode du bon objet est exécutée. C'est le principe du **polymorphisme**.

Étape 3 – Séparer le métier de l'affichage

Les classes `Point`, `Rectangle` et `Cercle` représentent maintenant correctement le métier. Nous voulons afficher ces objets dans une fenêtre sans ajouter de code Swing directement dans les classes métier.

1. Préparer l'étape 3. Copiez depuis `etape2_formes` vers `etape3_affichage` :

Point.java
Forme.java
Rectangle.java
Cercle.java

Ne copiez pas Main.java.

Le répertoire contient déjà les nouveaux fichiers liés à l’affichage :

Imprimable.java
Dessinable.java
RectangleAffichable.java
CercleAffichable.java
DrawUtils.java
AfficheurPanel.java
MainSwing.java

Les imports `java.awt`, `javax.swing` et `java.util` présents dans ces fichiers donnent accès à des classes de la bibliothèque standard. Vous n’avez pas à les modifier dans ce TP.

2. Deux interfaces pour l’affichage. Complétez `Imprimable.java` avec une méthode :

```
1 String imprimer();
```

Complétez `Dessinable.java` avec une méthode :

```
1 void dessiner(Graphics g);
```

3. Les adaptateurs d’affichage. Complétez `RectangleAffichable` et `CercleAffichable`.

Chaque objet affichable doit :

- conserver une référence vers la forme métier correspondante ;
- implémenter `Imprimable` ;
- implémenter `Dessinable` ;
- retourner une représentation textuelle dans `imprimer()` ;
- déléguer le dessin à la classe `DrawUtils` dans `dessiner(Graphics g)`.

Lisez les signatures de `DrawUtils.drawRectangle(...)` et `DrawUtils.drawCercle(...)` afin de déterminer quelles informations leur transmettre.

4. Le panneau Swing. La classe `AfficheurPanel` contient une liste de `Dessinable`. Complétez la boucle de `paintComponent(Graphics g)` pour demander à chaque objet de se dessiner.

Le constructeur utilise :

```
1 this.formes = List.copyOf(formes);
```

Expliquez pourquoi il s'agit d'une nouvelle liste mais **pas** d'une copie profonde des objets qu'elle contient.

5. Test final. Le fichier `MainSwing.java` crée les objets métier et prépare une fenêtre. Complétez ses `TODO`, puis compilez et exécutez :

```
javac *.java  
java MainSwing
```

Vous devez obtenir :

- une description textuelle du rectangle et du cercle dans le terminal ;
- une fenêtre contenant les deux formes dessinées.